

memor: architecture and product review

Reviewer framing: product architect, LLM memory and context optimisation. **Date:** 2026-08-22. **Method:** five parallel research agents (architecture, memory quality, competitive landscape, product positioning, context-engineering frontier), plus direct measurement against 2,465 local Claude transcripts (82.5M tokens) and the live memor database (33,970 artifacts, 4,133 recalls).

Every claim below cites a `path:line`, a command and its output, or a URL with the date read. Where a number is a lower bound or an estimate, it says so.

1. Verdict

memor is a **compression product with a memory feature**. The evidence says it should be a **compaction-integrity product with a compression feature**.

Three measurements, all taken on this machine, should redirect the roadmap:

Measurement	Result	Source
Large sessions (>2MB) that hit compaction	81.5% (22 of 27)	2,465 transcripts
User instructions lost to compaction	≥36.9% (433 of 1,172)	63 real compaction events
Compactions the compression hook avoids	0.107 per session	12 heaviest sessions
Recalls returning nothing	40.9% (1,690 of 4,133)	<code>recall_log</code>
...on projects with >2,000 artifacts	32.1%	<code>recall_log</code> × <code>artifacts</code>
Artifacts never once recalled	88.0% (29,909)	<code>memory_quality</code>

The compression work is finished and honest. It is capped at 3.9% of a request by arithmetic that four independent attacks failed to move (`docs/receiving-compressor-bottleneck.md`). Meanwhile the single event that destroys the most context value — compaction — is unmeasured, unaddressed, and happening in 4 out of 5 long sessions.

The strategic error is not the compressor. It is that the product measures the thing it can move rather than the thing that hurts.

2. What is genuinely excellent

2.1 The measurement discipline is the moat

This is not a compliment, it is a competitive assessment. In this session alone, seven estimates collapsed under measurement:

Estimate	First guess	Measured
Diff compressibility	39%	16%, then 0.1% realised
Session-start compression	30%	4.5%
Fetches-page squeeze	40%	7.4%, then 0.10% realised
Classifier release value	5.39%	0.10%
Exact-duplicate elision	1.78%	0.53%
Superseded file reads	15.77%	0.79%
"Code-safe" log crusher	3.53% at zero loss	circular; 8,371 lines still lost

Four documents in `docs/` are retractions of the author's own claims. The last of these is the most valuable artifact in the repository: a compressor that reported **zero** code loss was found to be measuring with the same regex it pinned with, and independent predicates showed it still dropped 8,371 indented code lines (`docs/receiving-compressor-bottleneck.md`).

No competitor does this. Mem0 publishes 26% over OpenAI on LOCOMO; Zep publishes 75.14% and argues Mem0's benchmark is flawed; Honcho publishes 60–90% token savings on its own eval site. Zep's most important observation is that **Mem0's own paper shows a plain full-context baseline beating Mem0** (~73% vs ~68%). Against that backdrop, a tool that ships `request-anatomy`, `hook-worth` and `eval-retention` so users can falsify its claims on their own traffic is genuinely differentiated.

The image-trap warning in `README.md:57–62` is the cleanest example: counting base64 put images at 31% of a session; they are 0.4%, because providers bill by dimensions. A 70× self-caught error, published.

2.2 The safety architecture is correct

The source guard is right, and the four attempts to weaken it this session all failed on evidence rather than nerve. Releasing guarded payloads to the log crusher destroys 5,230 code lines across 273 of 443 payloads, because `sed -n '1,200p' BalanceHero.tsx` and `git show rev:output.rs` are file reads that merely lack line numbers.

The system fails open almost everywhere, which is the right default for a sidecar: socket down falls back inline (`bin/memor-hook.py:80–85`), missing embedder returns the body unchanged (`memor/proxy/memory.py:115–120`), DB lock swallows and continues (`memor/store/sqlite_store.py:1483–1487`).

2.3 Concurrency is handled deliberately

WAL plus a 30-second busy timeout (`memor/store/sqlite_store.py:106`), with the motivating incident recorded in the comment: 356 "database is locked" errors in one session under Python's 5-second default.

3. What is weak

Ranked by measured impact.

3.1 Retrieval admits by recency, not relevance — this is the big one

The blended score is $0.50 \cdot \text{relevance} + 0.25 \cdot \text{recency} + 0.15 \cdot \text{kind} + 0.10 \cdot \text{quality}$ (`memor/retrieve/retriever.py`), with a 14-day recency half-life and an admission threshold of `0.3` (`memor/recall.py:120`).

Compute the score of a memory with **zero relevance**, created today, neutral quality, no kind boost:

$$0.50 \times 0 + 0.25 \times 1.0 + 0.15 \times 0 + 0.10 \times 0.5 = 0.300$$

That is exactly the threshold. The consequences are structural:

relevance	0d	7d	14d	30d	60d	120d
0.0	0.300 ✓	0.227	0.175	0.107	0.063	0.051
0.2	0.400 ✓	0.327 ✓	0.275	0.207	0.163	0.151
0.4	0.500 ✓	0.427 ✓	0.375 ✓	0.307 ✓	0.263	0.251
0.6	0.600 ✓	0.527 ✓	0.475 ✓	0.407 ✓	0.363 ✓	0.351 ✓

Read the top row: **a completely irrelevant memory is admitted for roughly two weeks purely because it is new**. Read down the 60-day column: after two months a memory needs 0.4+ relevance to be admitted at all.

Directly: an irrelevant memory from today (0.300, admitted) **outranks a 40%-relevant memory from 60 days ago** (0.263, rejected).

This explains both headline failures at once. It explains the 40.9% zero-hit rate, because anything older than a few weeks must clear a relevance bar that the recency term was sized to dominate. And it explains why the durable knowledge — the architecture decision from three months ago, which is exactly what memory is *for* — is the first thing evicted.

The counter-hypothesis, that stores are simply empty, is false. `ygo` has **5,002 artifacts and a 52.1% zero-hit rate**; across all projects with >2,000 artifacts the rate is still 32.1%.

The context-rot literature makes this worse than a missed opportunity. Chroma's 2025-07-14 report (18 models, 194,480 calls) finds **distractors are the dominant degradation lever and their harm amplifies with input length**. A memory system that admits irrelevant-but-fresh content is *manufacturing distractors* at the position where they measure most damaging.

3.2 Most of what is stored is not memory

Of 6,948 memory artifacts (5.67M tokens), **46.3% of tokens are provably ephemeral scaffolding**: 2,316 are subagent task prompts ("You are implementing Task 11..."), 293 are status reports, plus caveat banners and plan dumps. Sampling the remaining "possibly durable" half shows more of the same — work logs, research summaries, review checklists.

The signal that *does* matter: **52.4% of memories ever recalled carry rationale language** ("because", "instead of", "turned out", "root cause"), against 44.8% of the corpus. Rationale is what retrieval is already selecting for, and it is exactly the class the literature says is unrecoverable from a repo.

Anthropic reached the same conclusion independently. Claude Code's auto-memory docs state it "**skips anything it can derive from the codebase, such as architecture, file paths, or debugging fixes**" and store `project` memories only for decisions "*that Claude can't derive from the code or git history*". They encoded the thesis as a write-filter. `memor` has no such filter.

3.3 Finished work is stranded on unreachable tags

Four shelved tags, **1,759 insertions**, contained in no branch:

Tag	Commits	Diff
<code>shelved/temporal-validity</code>	11	19 files, +699
<code>shelved/reaffirmation-recency</code>	4	9 files, +532
<code>shelved/dashboard-service</code>	1	4 files, +352
<code>shelved/distillation-quality</code>	2	3 files, +176

`temporal-validity` is the painful one. The `disputes` table and the `validity` column exist in the live database — `disputes` holds 0 rows, `validity` is 1.0 across all 4,061 rows, and **neither string appears anywhere in the Python source**. The schema is a fossil of a completed feature that was built, tagged, and orphaned.

This matters because staleness is the known failure mode of code memory, and Zep built an entire company on temporal validity as the differentiator. `memor` built it and shelved it.

3.4 Dead code and a silent kill switch

`memor/proxy/output_shaper.py` is **271 lines referenced from nowhere** (confirmed: no match in any module outside itself).

More serious: `memor/proxy/shim.py:35-46` sets `compressor_ready = False` on any exception, and `prepare_request_body` checks that latch on entry. **One transient DB lock or malformed payload permanently disables compression for the life of the process**, with no reset path and no alert. A silently-degraded proxy is indistinguishable from a working one.

3.5 The evaluation is construct-invalid for the product

`memor eval-longmemeval` reports 95.0% any-hit on LongMemEval_S. The harness scores **set membership of session IDs** (`memor/eval/longmemeval.py:111`), not answers — the file's own docstring admits "any-hit overstates how many questions the agent could actually answer".

Worse, LongMemEval is conversational personal-assistant memory ("I have a dog named Max"). `memor` is a coding-agent memory. The one end-to-end number in the repository is a collapse: the counterfactual win rate read **63.8% before the harness was corrected to call production `recall()`**, and **8.6% after** (`CHANGELOG.md:39`).

So the memory half has never been evaluated on the thing it claims to do, using the same rigour that has been applied to compression seven times over.

4. The strategic risk

4.1 The platform has already shipped most of the compression story

Capability	Date	Evidence
Context editing (<code>clear_tool_uses_20250919</code>)	2025-09-29	anthropic.com/news/context-management
Memory tool (<code>memory_20250818</code>), all Claude 4+	2025-09-29	docs.anthropic.com/.../memory-tool
Claude Code auto-compact, on by default	shipped	code.claude.com/docs/en/costs
Claude Code auto-memory	shipped	docs.claude.com/en/docs/claude-code/memory
Cursor Memories (beta)	2025-06-04	cursor.com/changelog/1-0

(All read 2026-08-22.)

Anthropic reports context editing alone at **+29%**, with memory at **+39%**, and **84% token reduction** on a 100-turn web-search eval. That 84% is a different denominator — tool-result-dominated traffic, exactly what memor's own README warns about — but a prospective user will compare 84% to 11.7% and stop reading.

Leading with compression means competing against a free, default-on platform feature on its strongest number, using your weakest one.

4.2 What the platform deliberately left open

Anthropic ships the memory *protocol* and a reference filesystem handler, and states plainly that "memory lives entirely in your application". They specified the socket, not the appliance. What goes into memory, when, deduplicated how, ranked by what — is unowned.

Three durable positions remain:

1. **Curation policy.** The unowned half of the memory tool.
2. **Cross-harness portability.** A developer using Claude Code, Codex and Cursor has three memory silos. No provider will make its memory work well inside a rival's harness. Every serious competitor leads with this.
3. **Measurement.** No platform will ship a tool that prints how little its own optimisation saved.

4.3 The economics were always wrong for this user

Claude Code's own docs: "Claude Max and Pro subscribers have usage included in their subscription, so the session cost figure isn't relevant for billing purposes." On a Max plan, every compression percentage converts to **\$0.00**.

Even for the API user, memor's honest 0.8–3.9% blended saving against the documented \$150–250/developer/month is **\$2–10/month** — below the threshold of a procurement conversation, a security review, or the annoyance of installing a proxy.

The currency is context headroom. The product measures dollars.

5. Where the product should go

5.1 Compaction is the correct target

Compaction is simultaneously the highest-risk correctness moment and the largest cache-invalidation event in an agent session. The literature is now specific:

- **COMPINT** (arXiv:2608.11242, 2026-07-31): current compactors retain **17% of user-issued session constraints**, and most perform worse than not compacting at all. Their fix — a **sidecar extractor alongside the compactor** — reaches **>90% retention without modifying the compactor or the model**.
- **ConstraintRot** (arXiv:2606.22528, 2026-06-21): constraint violation rises from **0% in full context to 30% after compaction, 59% for some models**. The decomposition is decisive: **when the constraint survives the summary, violation stays 0%; when dropped, it reaches 38%**. "Constraint Pinning" restores 0%, training-free.
- **Parallel Context Compaction** (arXiv:2605.23296): compaction is *non-deterministically* lossy; prompt instructions to the compactor "are largely ignored".
- **Copilot production traces** (arXiv:2608.00101, 3.2M users): KV cache hit rate ~90% within a turn, 55% across turns, "drastically invalidated" after compaction.

Anthropic documents the hole in their own product. Their "What survives compaction" table lists path-scoped rules and nested CLAUDE.md as **"Lost until a matching file is read again"**, and the memory doc states: *"If an instruction disappeared after compaction, it was given only in conversation."* Their recommended fix is for the user to go write it in CLAUDE.md by hand.

That is a gap a sidecar can close automatically, and COMPINT proves the shape works.

5.2 It reproduces on this machine

Measured over 63 real compaction events in the local corpus, extracting human-typed constraints from turns preceding each compaction and testing whether they survived into the summary:

```
human-typed constraints found : 1,172
retained in summary           : 739 (63.1%)
LOST                          : 433 (36.9%)
```

Examples lost verbatim:

- *"yes please we don't want to over engineer something that is a simpler fix right ?"*
- *"When we moved from YRS to YGO didnt we try to make sure about the feature and information parity..."*
- *"If still pending after 60 minutes total runtime, something is wrong — report to user and stop the wakeup loop"*

36.9% is a lower bound. The retention test counts a constraint as surviving when 60% of its content words appear in the summary, and manual inspection shows that fires on stopword overlap ("will", "with", "your") where the summary merely discusses the topic without preserving the instruction. True loss is higher.

This is a smaller effect than COMPINT's 83% loss, and the honest reading is that Claude Code's compactor is better than the average in that study — but a third of the user's own instructions vanishing is still an order of magnitude larger than anything compression can recover.

6. Ranked backlog

Ranked by (measured impact × confidence), not by ease.

P0 — Constraint pinning

What. Detect user-issued constraints in conversation, persist them outside the context, re-inject after every compaction event.

Why first. It is the only item on this list where the pain is measured locally ($\geq 36.9\%$), the mechanism is published, the precedent is proven at the same architectural shape (COMPINT: 17%→90%), and the vendor has documented the gap in its own manual.

Success metric. Constraint retention across compaction, measured by replaying the local corpus: from 63.1% (optimistic baseline) to >90%. This is falsifiable on data that already exists.

Effort. Moderate (estimate). Detection is the hard part; injection reuses the existing UserPromptSubmit path.

Risk. Detection precision. A false positive pins noise into every subsequent turn, which per §3.1 manufactures a distractor. Ship with a conservative detector, measure precision before recall.

P1 — Rebalance retrieval admission

What. Decouple the admission threshold from the recency term. Options: raise the threshold above the fresh-irrelevant score of 0.300, or gate on raw relevance before blending, or lower `w_rec`.

Why. §3.1. It is a small change with two large effects: fewer distractors injected, and durable old knowledge stops being evicted for being old.

Success metric. Zero-hit rate on stores with >2,000 artifacts falls from 32.1%; injected-memory relevance rises. Both computable from `recall_log`.

Effort. Small. Constants plus a re-run of the eval harness.

Risk. Regression on the LongMemEval number, which is the wrong benchmark anyway. Build P2 first if you want a trustworthy signal.

P2 — A coding-memory benchmark

What. Replace LongMemEval-as-headline with an eval built from local sessions: given a question answerable only from a past session, does recall surface the right artifact, and does the agent then answer correctly?

Why. The memory half has never faced the rigour the compression half faced seven times. The one end-to-end number went 63.8% → 8.6% when the harness was corrected. Every subsequent memory decision is uncalibrated until this exists.

Success metric. The benchmark existing and producing a number the author believes.

Effort. Moderate. The corpus (2,465 sessions) and grading precedent (`eval-retention`) both exist.

P3 — Write-side curation filter

What. Refuse to store what the repo already answers. Prefer rationale, failed attempts, corrections, preferences.

Why. 46.3% of memory tokens are ephemeral scaffolding; 88% of artifacts are never recalled. Anthropic independently shipped this as a write-filter.

Success metric. Never-recalled share falls from 88%; rationale share of stored tokens rises from 44.8%.

Effort. Small-to-moderate; `_signal_score()` already exists as the hook point (`memor/ingest/claude_code.py:82`).

P4 — Unshelve temporal validity

What. Restore `shelved/temporal-validity` (11 commits, +699) or delete the orphaned schema.

Why. Staleness is the known failure mode of code memory. The work is done. Leaving `disputes` and `validity` in the live schema with no code is a trap for the next reader.

Effort. Small if it merges; the decision is whether it earns its complexity.

P5 — Repair the operational faults

- Remove the `compressor_ready` latch or give it a reset (`memor/proxy/shim.py:35-46`).
- Delete `memor/proxy/output_shaper.py` (271 dead lines).
- Extract `memor/install/` from `memor/cli.py` (1,871 lines; install logic for seven agents at lines 701–1102).

7. What not to build, and what is still unknown

Do not build

- **More compression.** Four independent attacks measured negative in one day. The remaining 4.04% is source code, and the only things to do with source code are pass it through or corrupt it. `79702be` already records this.
- **A better content classifier.** Worth 0.03%–4.04% depending only on which compressor receives the release. The classifier was never the bottleneck.
- **More integration surfaces.** Nine agents across three mechanisms with two non-identical support matrices. Each new surface is maintenance paid in silent breakage.

Open questions

1. **Does constraint pinning survive contact with precision?** The P0 risk is that a detector good enough to catch 90% of constraints also pins enough noise to hurt. Unmeasured.
2. **Is the local 36.9% representative?** It is one user, one harness, 63 compaction events. COMPINT reports far worse across compactors; Claude Code may be unusually good.
3. **Does memory help at all, end-to-end?** The honest answer today is that nobody knows, because the only end-to-end measurement collapsed from 63.8% to 8.6% under correction. P2 exists to answer this.
4. **Would rebalancing retrieval fix zero-hit, or expose that the artifacts are not worth retrieving?** §3.2 suggests the latter is partly true.
5. **How much does compaction cost in cache invalidation on this machine?** Copilot's traces say hit rate is "drastically invalidated"; memor already reads usage from the response stream and could measure it.

Appendix: what was measured, and how

Claim	Method
81.5% of >2MB sessions compact	Scanned 2,465 transcripts for <code>isCompactSummary</code> / continuation markers, bucketed by size
≥36.9% constraint loss	Extracted human-typed constraint sentences from turns preceding 63 compactions; tested 60% content-word survival into the summary. Boilerplate excluded after a first pass wrongly counted 21,421 skill-file lines
0.107 compactions avoided	Ran the real hook over the 12 heaviest compacting sessions; 256,084 tokens removed ÷ 200K window ÷ 12
40.9% / 32.1% zero-hit	<code>recall_log</code> grouped by status, joined to per-project artifact counts
88% never recalled	<code>artifacts</code> minus <code>memory_quality</code>
46.3% ephemeral	Regex classification of all 6,948 memory artifacts by opening shape
52.4% rationale in recalled	Rationale-language regex over recalled vs total
Admission table	Direct evaluation of the scoring formula from <code>retriever.py</code> constants
Dead code, orphaned schema	<code>grep</code> across the package; <code>git tag</code> / <code>git branch --contains</code>

Research agents: architecture (`arch-map2`), memory quality (`memory-qual2`), competitive landscape (`landscape`), product positioning (`product-pos`), context frontier (`ctx-frontier`). Their full reports are in `docs/competitive-landscape-2026-08.md` and `docs/research/context-quality-2026-08.md` .